

AIRLINE TICKET BOOKING MANAGEMENT SYSTEM

VITyarthi – Build Your Own Project

Course: Programming in Java (CSE2006)

Student Name: _____

Registration Number: _____

Faculty / Section: _____

Command-line Java application demonstrating OOP, collections, exception handling, multithreading and I/O streams.

1. Introduction

The Airline Ticket Booking Management System is a small terminal-based Java application for demonstrating a realistic booking workflow. A user can view flights, search a route, book a ticket, view booking history, and cancel a booking. The project uses local object serialization for persistence, so it can be executed from a command line without a GUI or external server.

The VITyarthi project instructions require an original project aligned with the subject, at least three functional modules, clear input/output, at least four non-functional requirements, modular implementation, 5–10 meaningful classes/files for coding projects, testing, and documentation/design artefacts. (VITyarthi Project Instructions)

2. Problem Statement

Manual ticket-booking demonstrations often require separate steps for checking flight availability, entering passenger details, maintaining booking records, and updating seats. This project provides a single command-line workflow that combines these operations while applying core Java programming concepts.

Scope limitation: this is an academic simulation. It does not connect to live airline inventory, process real payments, or generate legally valid airline tickets.

3. Objectives

- Build a simple real-world Java application.
- Apply object-oriented design through classes, inheritance and encapsulation.
- Use Java collections for in-memory management.
- Demonstrate exception handling and input validation.
- Demonstrate multithreading through booking confirmation notification.
- Use Java I/O streams for persistent local storage.

4. Functional Requirements

Module	Requirement
Flight Management	Display all available flights with route, departure, fare and seat count.
Flight Search	Search flights using source and destination.
Booking Management	Enter passenger details, validate input, create booking ID, reduce seat count and save data.
Booking History	Display all stored bookings and their status.
Cancellation	Cancel an active booking and release the associated seat.
Notification	Start a separate Java thread to print booking confirmation notification.

5. Non-functional Requirements

Requirement	Description
Usability	Menu-driven terminal interface with simple prompts.
Reliability	Invalid inputs and failed operations produce controlled error messages.
Maintainability	Responsibilities are split across multiple classes.
Portability	Requires only Java JDK 17+ and a terminal.
Resource Efficiency	Uses local file storage and standard Java collections; no external server is needed.
Error Handling	Custom ValidationException plus try-catch handling.

6. System Architecture

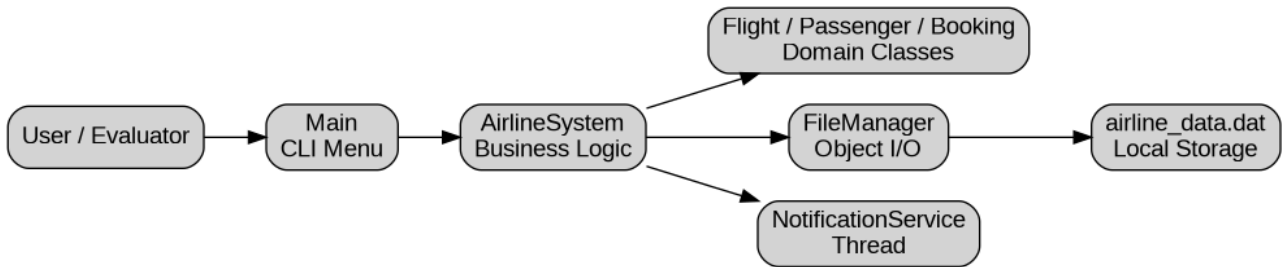


Figure 1. High-level system architecture.

The Main class handles terminal interaction. AirlineSystem contains the booking business logic. Flight, Passenger and Booking represent domain data. FileManager handles persistence through Java object streams, while NotificationService demonstrates multithreading.

7. Design Diagrams



Figure 2. Use case diagram.

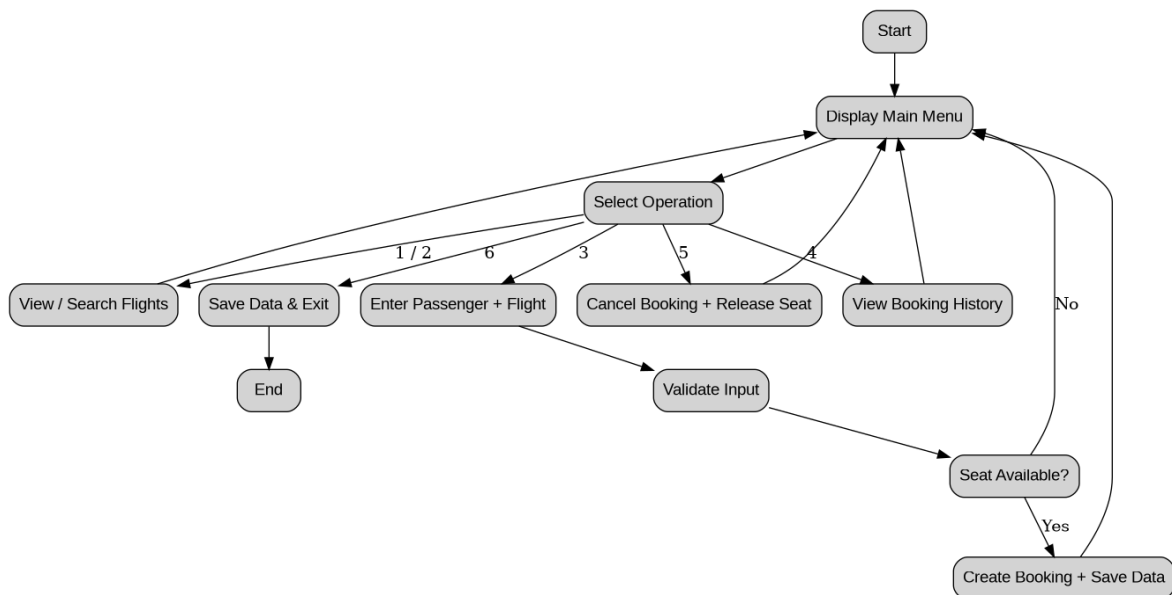


Figure 3. Booking workflow.

7.1 Class Diagram

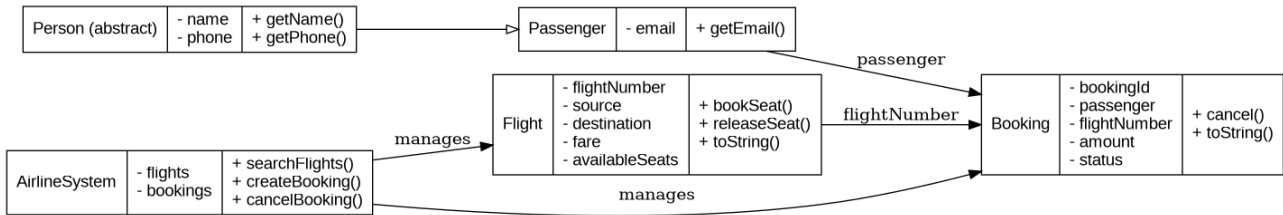


Figure 4. Main class relationships.

7.2 Sequence Diagram

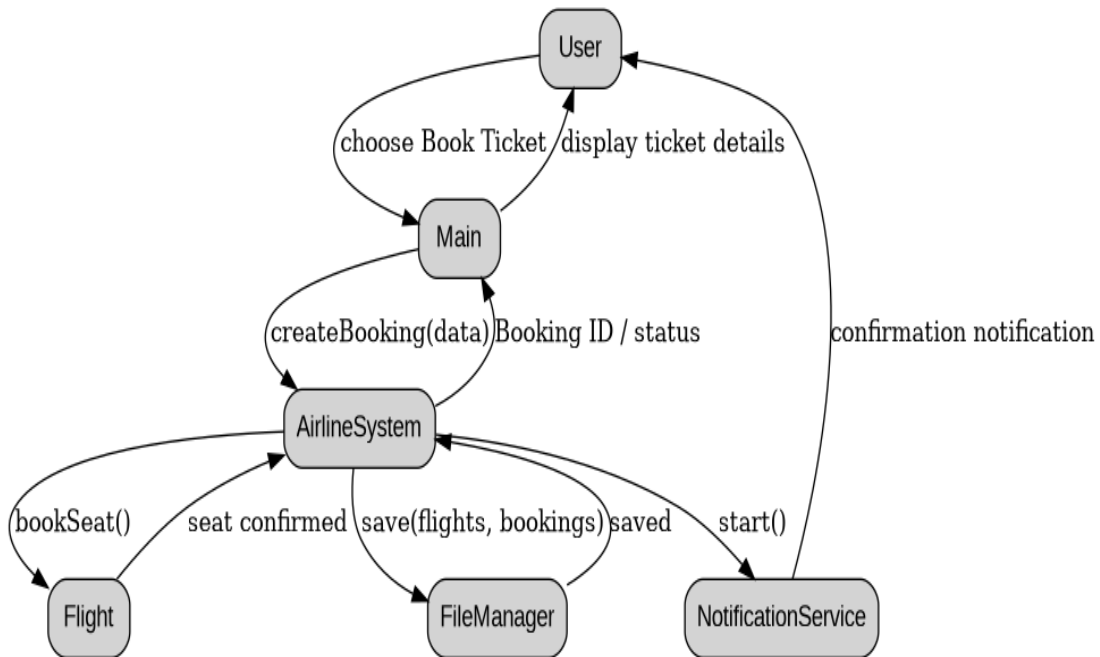


Figure 5. Ticket-booking sequence.

7.3 Data / Storage Design

The application uses an object-serialization file instead of a relational database to keep the project simple and terminal-ready. The following conceptual model shows the stored relationships.

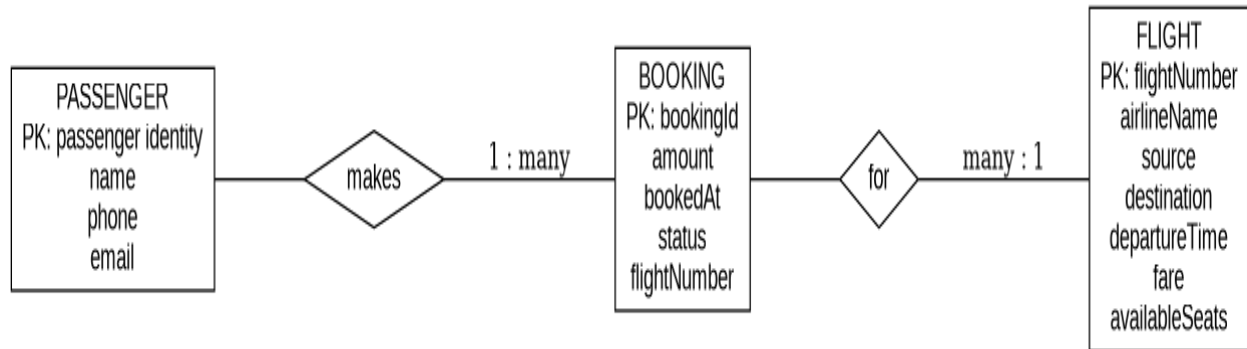


Figure 6. Conceptual data model for the stored objects.

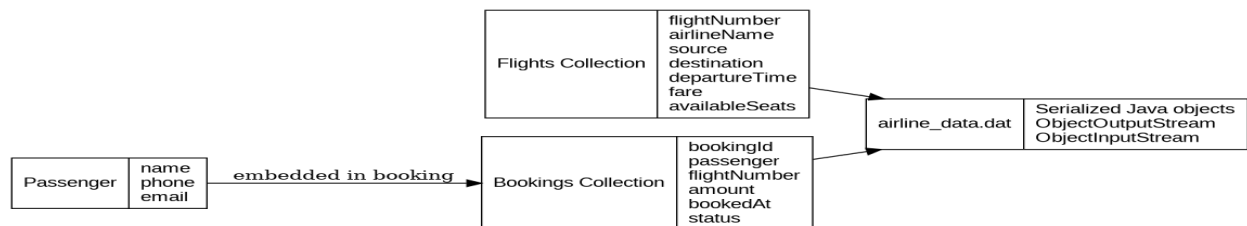


Figure 7. Local storage design.

8. Design Decisions & Rationale

Command-line interface: The project is intentionally simple and can be executed entirely from a terminal, matching the requirement for command-line executability.

ArrayList: Suitable for a small academic application and directly demonstrates the Java Collections Framework.

Inheritance: Passenger extends Person to demonstrate an is-a relationship and reuse common passenger identity fields.

Object serialization: Keeps persistence simple and avoids external database configuration.

Thread: NotificationService implements Runnable and is started after a successful booking to demonstrate multithreading.

9. Implementation Details

The project contains ten Java classes/files in the airline package. Main provides the menu; AirlineSystem coordinates business operations; Flight, Person, Passenger and Booking model the domain; FileManager provides I/O; InputValidator and ValidationException handle validation; NotificationService demonstrates a background thread.

The course syllabus includes Java OOP concepts such as classes, constructors, inheritance, method overriding, polymorphism and encapsulation; it also includes exception handling, multithreading, collections and I/O streams. (CSE2006 Programming in Java syllabus, Unit 2–5)

10. Screenshots / Results

```
Screenshot 1 - Main Menu and Flight List

=====
AIRLINE TICKET BOOKING MANAGEMENT SYSTEM
=====

----- MAIN MENU -----
1. View all flights
2. Search flights
3. Book ticket
4. View bookings
5. Cancel ticket
6. Exit
Enter your choice: 1

----- AVAILABLE FLIGHTS -----
Flight   Airline      From      To      Departure   Fare      Seats
AI101    Air India    Delhi     -> Mumbai  08:30      Rs.5200.00 25 seats
6E205    IndiGo       Mumbai    -> Bengaluru 11:15      Rs.4300.00 30 seats
UK810    Vistara      Delhi     -> Bengaluru 16:45      Rs.6100.00 20 seats
IX332    Air India Exp Kochi     -> Delhi    19:20      Rs.5800.00 18 seats
```

Figure 8. Main menu and flight list.

```
Screenshot 2 - Flight Search

=====
FLIGHT SEARCH
=====
Enter source: Delhi
Enter destination: Bengaluru

Search results:
UK810    Vistara      Delhi     -> Bengaluru 16:45      Rs.6100.00 20 seats

Search completed successfully.
```

Figure 9. Flight search.

Screenshot 3 - Successful Ticket Booking

```
=====
                        TICKET BOOKING
=====
Enter flight number: UK810
Passenger name: Archit Sharma
Phone: 9876543210
Email: archit@example.com

Ticket booked successfully!
Booking ID : B0001
Passenger  : Archit Sharma
Flight     : UK810
Amount    : Rs. 6100.00
Status    : CONFIRMED

[Notification Thread] Confirmation sent to Archit Sharma for booking B0001.
```

Figure 10. Successful ticket booking.

Screenshot 4 - Booking History and Cancellation

```
=====
                        BOOKING HISTORY / CANCEL
=====
```

Booking	Passenger	Flight	Amount	Status
B0001	Archit Sharma	UK810	Rs.6100.00	CONFIRMED

```

Enter booking ID to cancel: B0001
Booking cancelled successfully.

Booking    Passenger      Flight    Amount    Status
B0001     Archit Sharma    UK810    Rs.6100.00  CANCELLED
```

Figure 11. Booking history and cancellation.

11. Testing Approach

Testing was performed through terminal execution. Functional cases cover normal and invalid flows: viewing flights, valid and invalid search, valid booking, empty passenger input, booking history, valid cancellation, invalid cancellation, persistence after restart, and the confirmation thread. A detailed test table is included in TESTING.md.

Compilation command:

```
javac -d out src/airline/*.java
```

12. Challenges Faced

The main implementation challenges were maintaining consistent seat counts after booking/cancellation, preserving booking data between program runs, validating user input, and demonstrating multithreading without making the menu difficult to use.

13. Learnings & Key Takeaways

The project provides practical experience with class design, inheritance, encapsulation, collections, exception handling, object streams, package organization, and basic multithreading. It also demonstrates how several individual Java concepts can be combined into a small business-oriented application.

14. Future Enhancements

- Add JDBC with SQLite/MySQL for relational persistence.
- Add login and role-based access.
- Add date-wise flight schedules and seat numbers.
- Add payment simulation and invoice generation.
- Add a GUI or web interface.
- Add JUnit tests and automated build configuration.
- Integrate a real airline API only in a future non-academic version.

15. References

1. VITyarthi, "Build Your Own Project – General Project Instructions & Submission Guidelines," provided project instruction document. (VITyarthi Project Instructions)
2. VIT Bhopal University, CSE2006 Programming in Java syllabus, uploaded course document. (CSE2006 Programming in Java syllabus)
3. Herbert Schildt, *Java: The Complete Reference*, 11th Edition, Oracle Press, 2018. Listed in the course syllabus.

End of Report